🖥️ SchedOS — Full Project Structure & File Guide

> **Note:** This document is kept locally on your machine to help you understand every file and easily explain the project to your teacher or viva examiner.

---

1. 🛠️ Tools & Frameworks Used

Here is the simple list of tools and technologies used to build this website:

Tool / Technology	What it is	Why we use it in SchedOS
React 19	Modern JavaScript UI library	Builds the website interface using interactive, reusable components.
TypeScript	JavaScript with types	Makes the code clean, prevents bugs, and ensures data (like burst times and arrival times) is always accurate.
Vite	Super-fast build tool	Runs the local development server in seconds and compiles the project for production.
Tailwind CSS v4	Utility styling framework	Styles the entire app with a retro-terminal computer look (CRT scanlines, monospace fonts, clean borders).
Tesseract.js	Optical Character Recognition (OCR)	Reads photographs or screenshots of process tables and automatically extracts process data.
Recharts	Interactive chart library	Draws the bar charts on the Compare page to compare Waiting Time, Turnaround Time, etc.
PrismJS	Code syntax highlighter	Displays clean, color-coded code examples in C, Python, and Bash with line numbers.
Lucide React	Icon library	Provides clean icons for buttons (Play, Pause, Step, Upload, Reset, etc.).
Vitest	Testing framework	Runs 557 automated unit tests to prove every scheduling algorithm is 100% mathematically correct.

Tool / Technology	What it is	Why we use it in SchedOS
vite-plugin-singlefile	Vite bundling plugin	Packages the entire website (HTML, CSS, JavaScript) into a single standalone `index.html` file that can open offline in any browser without needing a server.

---

2. 📂 Project Folder Structure Overview

```
SchedOS/
├── Assets/                 # Flowcharts, screenshots, and diagrams for report & viva
├── dist/                   # Production build output (single standalone index.html)
├── node_modules/           # Installed third-party packages
├── src/                    # All source code for the website
│   ├── algorithms/         # The 8 CPU scheduling algorithm implementations
│   ├── codeExamples/       # Exam-ready code in C, Python, and Bash
│   ├── components/         # Reusable UI pieces (Gantt chart, table, controls, etc.)
│   ├── data/               # Pre-made sample datasets
│   ├── engine/             # The brain: calculations, metrics, parsing, validation
│   ├── pages/              # The main pages (Home, Visualizer, Compare, etc.)
│   ├── test/               # 557 automated unit tests
│   ├── types/              # Data models and TypeScript interfaces
│   ├── App.tsx             # Main page container and state manager
│   ├── main.tsx            # Website entry point
│   └── styles.css          # Theme styles, CRT effects, and fonts
├── index.html              # HTML shell of the web app
├── package.json            # Project dependencies and script commands
├── tsconfig.json           # TypeScript configuration
├── vite.config.ts          # Vite build tool configuration
└── SchedOS_Project_Report.docx # Complete laboratory project report
```

---

3. 📄 Detailed File-by-File Summary

🧠 `src/engine/` — The Core Brain of SchedOS

This folder contains the core logic. It does all the math and scheduling without touching the UI.

File Name	Short Easy Summary
`scheduler.ts`	**The Simulation Runner:** Takes your processes and the chosen algorithm, then runs the simulation clock tick-by-tick. It handles when processes arrive, when they finish, and when they are preempted.

File Name	Short Easy Summary
`metrics.ts`	**The Math Calculator:** Calculates the golden OS formulas: Completion Time (CT), Turnaround Time ($TAT = CT - AT$), Waiting Time ($WT = TAT - BT$), Response Time ($RT = FirstStart - AT$), CPU Utilization (%), and Throughput.
`datasetParser.ts`	**The Smart Text & OCR Reader:** Reads text from the Paste Box or OCR results. It understands key-values (`AT=0 BT=5`), tables (`P1 0 5`), natural sentences, bullet points (`•`), and automatically cleans common OCR reading mistakes.
`timeline.ts`	**The Animation Frame Generator:** Breaks down the simulation into tick-by-tick steps so the user can hit Play, Step Forward, or Step Backward to watch it like a movie.
`validation.ts`	**The Error Checker:** Checks the user's input before running. It flags errors like empty IDs, negative arrival times, or burst times less than 1.
`comparison.ts`	**The Multi-Algorithm Judge:** Runs the same dataset through multiple algorithms, collects their stats, picks the winner (lowest WT), and writes natural language explanations of why one algorithm beat another.

⚙️ `src/algorithms/` — The 8 CPU Scheduling Algorithms

Each file here implements one specific scheduling policy and generates the Gantt blocks.

File Name	Algorithm Type	Short Easy Summary
`fcfs.ts`	Non-Preemptive	**First-Come, First-Served:** Runs processes in the order they arrive. Once a process starts, it keeps the CPU until it finishes.
`sjf.ts`	Non-Preemptive	**Shortest Job First:** When the CPU is free, picks the waiting process with the smallest burst time.
`srtf.ts`	Preemptive	**Shortest Remaining Time First:** Like SJF, but if a new process arrives with a shorter burst than what's left of the current process, it interrupts (preempts) immediately.
`roundRobin.ts`	Preemptive	**Round Robin:** Gives each process a fixed slice of time (Time Quantum). When time expires, it moves to the back of the queue.

File Name	Algorithm Type	Short Easy Summary
`priorityNonPreemptive.ts`	Non-Preemptive	**Priority (Non-Preemptive):** Picks the process with the highest priority and runs it until completion. Supports both "lower number = higher priority" and "higher number = higher priority".
`priorityPreemptive.ts`	Preemptive	**Priority (Preemptive):** If a process with higher priority arrives, it immediately pauses the current process and runs the new one.
`ljf.ts`	Non-Preemptive	**Longest Job First:** Picks the process with the biggest burst time. Used to study bad convoy effects.
`lrtf.ts`	Preemptive	**Longest Remaining Time First:** Always gives the CPU to the process with the most work left, balancing work across processes with frequent switches.
`index.ts`	Registry	Gathers all 8 algorithms into a single list with their names, tags, and default settings.

💻 `src/pages/` — The Main Application Screens

These are the 6 visible views of the website.

File Name	Page Name	Short Easy Summary
`Home.tsx`	Dashboard / Home	Welcome landing page. Shows an animated CPU chip, interactive cards for algorithms, and a 5-step guided walkthrough for beginners.
`Visualizer.tsx`	Visualizer	**The main workspace:** Lets users enter processes, select an algorithm, watch the live Gantt chart, see the classroom table burst cuts, and inspect the final metrics table.
`Compare.tsx`	Compare Page	Lets users select 2 to 8 algorithms and runs them on the exact same dataset side-by-side. Displays comparison tables, trophy winner badges, bar charts, and written observations.
`Algorithms.tsx`	Algorithm Reference	A digital textbook explaining all 8 algorithms with rules, step-by-step summaries, advantages, disadvantages, and tie-breaking policies.

File Name	Page Name	Short Easy Summary
`CodeExamples.tsx`	Exam Code Examples	Provides 24 complete, working code programs in **C**, **Python**, and **Bash** for lab exams, plus a formula cheat sheet and common viva exam traps.
`About.tsx`	About SchedOS	Explains the project purpose, architecture, priority conventions, full math formulas, and creator credits.

🧩 `src/components/` — Reusable User Interface Pieces

These are the building blocks that make up each page.

File Name	What It Does (Short Easy Summary)
`GanttChart.tsx`	Draws the colorful Gantt timeline showing exactly which process ran at what time, with idle CPU hatched areas and tooltip inspection.
`ReadyQueue.tsx`	**Classroom Table Simulation:** Shows the process table with strikethroughs on remaining burst times (e.g., ~~5~~ → ~~3~~ → 0), exactly how students solve problems in university exams. Also shows Ready, Unarrived, and Finished queue lanes.
`ProcessTable.tsx`	The interactive input table where users can type PIDs, Arrival Times, Burst Times, and Priorities, add/remove rows, or choose a priority rule.
`DatasetImportModal.tsx`	The pop-up modal for importing data. Supports: (1) uploading an image/screenshot with automatic OCR, and (2) pasting text in multiple formats with an editable preview table.
`SimulationControls.tsx`	Media player control bar with Play, Pause, Step Forward, Step Backward, Reset, and a 0.5x–4x Speed slider.
`CPUVisualizer.tsx`	A hardware chip readout showing the active process, CPU clock tick, utilization percentage, and a live progress gauge.
`PerformanceCards.tsx`	Metric cards showing Average Turnaround Time, Average Waiting Time, Average Response Time, CPU Utilization, and Throughput.
`PerformanceSection.tsx`	Combines the performance cards and the sortable per-process metrics table into one neat section.

File Name	What It Does (Short Easy Summary)
`MetricsTable.tsx`	A clean, sortable table listing each process's CT, TAT, WT, and RT with color-coded badges.
`ComparisonChart.tsx`	Interactive bar chart built with Recharts comparing different algorithms side-by-side on any metric.
`CodeViewer.tsx`	Syntax-highlighted code block using PrismJS with line numbers and a one-click copy button.
`SystemLog.tsx`	Monospace terminal box that prints every event (arrival, dispatch, preemption, completion) as it happens in real time.
`AlgorithmSelector.tsx`	Button group allowing the user to switch between the 8 scheduling algorithms.
`Sidebar.tsx`	Left navigation bar with quick links to all 6 pages and active tab indicators.
`HeroAnimation.tsx`	Animated interactive CPU circuit diagram shown on the home page.
`EmptyState.tsx`	A friendly message displayed when no data has been entered yet.
`Panel.tsx`	A styled card container with a retro-terminal title bar and border.
`processColors.ts`	Automatically gives each process (P1, P2, P3...) a distinct, high-contrast color for the Gantt chart.
`useSimulation.ts`	Custom React hook that controls the simulation playback loop and keyboard shortcuts (Space to pause, arrow keys to step).
`useHeroCycle.ts`	Custom React hook that drives the animated CPU cycle on the home page.

💻 `src/codeExamples/` — Polyglot Exam Implementations

Contains 24 full, working programs with user input handling:

- `fcfs.ts` — Full programs in C, Python, and Bash for FCFS

- `sjf.ts` — Full programs in C, Python, and Bash for SJF
- `srtf.ts` — Full programs in C, Python, and Bash for SRTF
- `roundRobin.ts` — Full programs in C, Python, and Bash for Round Robin
- `priorityNonPreemptive.ts` — Full programs in C, Python, and Bash for Non-Preemptive Priority
- `priorityPreemptive.ts` — Full programs in C, Python, and Bash for Preemptive Priority
- `ljf.ts` — Full programs in C, Python, and Bash for LJF
- `lrtf.ts` — Full programs in C, Python, and Bash for LRTF
- `index.ts` — Combines all code examples and adds memory tips, formulas, and viva traps

---

🧪 `src/test/` — Automated Unit Tests

Ensures every single part of the engine is tested and proven correct.

Test File	What It Tests	Total Tests
`algorithms.test.ts`	Verifies all 8 algorithms against edge cases: simultaneous arrivals, idle gaps, preemption, quantum slicing, and tie-breaking.	**539 tests**
`datasetParser.test.ts`	Verifies text parsing, OCR error correction (e.g., `1"` → `11`), bullet point stripping (`•`), CSV, Markdown tables, and natural language.	**15 tests**
`metrics.test.ts`	Verifies mathematical relationships: $TAT = CT - AT$, $WT = TAT - BT$, and averages.	**3 tests**
Total Automated Tests	All passing with Vitest	**557 tests**

📁 Root Files & Configuration

File Name	Description
`App.tsx`	Top-level React component. Holds the active page state and coordinates global lab settings.
`main.tsx`	The startup file that attaches the React application to the browser's DOM.
`index.html`	The base HTML template containing font links and the root mount point.
`styles.css`	Tailwind CSS v4 styling rules, custom scrollbars, monospace typography, and retro CRT scanlines.

File Name	Description
`package.json`	Defines project name, scripts (`npm run dev`, `npm run build`, `npm test`), and dependencies.
`vite.config.ts`	Configures Vite plugins (`@vitejs/plugin-react`, `@tailwindcss/vite`, `vite-plugin-singlefile`).
`tsconfig.json`	Strict TypeScript compiler settings to prevent runtime bugs.
`SchedOS_Project_Report.docx`	The full academic lab report for university submission.
`Assets/`	Contains flowchart images (`Flowchart_1_System_Flow.png`, etc.) and UI screenshots.

---

4. 🚀 Common Commands Quick Reference

Command	What it does
`npm run dev`	Starts the local development server at `http://localhost:5173`
`npm run build`	Builds the complete project into a single offline file (`dist/index.html`)
`npm test`	Runs all 557 automated unit tests with Vitest
`npm run preview`	Previews the production build locally

---

5. 💡 How to Explain SchedOS to Your Teacher in 3 Minutes

1. **Introduction:**
   "SchedOS is an interactive CPU scheduling simulator built with React 19, TypeScript, and Tailwind CSS. It is designed to solve the problem students face when trying to visualize how the CPU schedules processes."

2. **The Key Innovation (Two Views Combined):**
   "Unlike other simulators that only show a Gantt chart, SchedOS combines the Gantt chart with a Classroom Table Simulation. As the clock ticks, remaining burst times are crossed out with strikethroughs (~~5~~ → ~~3~~ → 0), exactly how we solve exam problems on paper."

3. **Smart Dataset Import:**
   "Students can enter data manually, paste from Word, CSV, or Markdown tables, or upload a photo of a handwritten or printed process table using automated OCR (Tesseract.js) with image preprocessing."

4. **All 8 Algorithms & Code Reference:**
   "It covers all 8 standard CPU scheduling algorithms (preemptive and non-preemptive), features a side-by-side comparison page with bar charts and analytical observations, and includes exam-ready implementations in C, Python, and Bash."

5. **Proven Accuracy:**
   "The entire engine is verified by 557 automated unit tests with zero discrepancies."